

```
pip install google-genai
from google import genai
import os

client = genai.Client(
    api_key="enter api key"    #AQ.Ab8RN6K99L0YiYWHxvTv_ShF98y4ZIx0eS1pliD7gB3lnEwQ
)

prompt = """
Classify the sentiment of the following review:

'The laptop performance is excellent and battery life is amazing.'

Output only:
Positive, Negative, or Neutral
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print(response.text)
```

Positive

```
from google import genai
import os

client = genai.Client(
    api_key="api key"    #AQ.Ab8RN6K99L0YiYWHxvTv_ShF98y4ZIx0eS1pliD7gB3lnEwQ
)

prompt = """
Classify sentiment.

Example 1:
Review: The product is amazing.
Sentiment: Positive

Example 2:
Review: The service was terrible.
Sentiment: Negative

Example 3:
Review: The experience was okay.
Sentiment: Neutral

Now classify:

Review: The laptop performance is excellent and battery life is amazing.

Sentiment:
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print(response.text)
```

Sentiment: Positive

```
from google import genai
import os

client = genai.Client(
    api_key="api key"    #AQ.Ab8RN6K99L0YiYWHxvTv_ShF98y4ZIx0eS1pliD7gB3lnEwQ
)

prompt = """
A company has 125 employees.

40% work in Software Development.
25% work in Testing.
The remaining employees work in Support.
```

Think step by step and calculate:

1. Number of Software Developers
2. Number of Test Engineers
3. Number of Support Staff

"""

```
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print(response.text)
```

[Show hidden output](#)

```
from google import genai
import time

client = genai.Client(
    api_key="enter api key"    #AQ.Ab8RN6K99L0YiYWHaxvTv_ShF98y4ZIx0eS1pliD7gB3lnEwQEY
)

prompt = """
Summarize the text in exactly 3 bullet points.

Artificial Intelligence is transforming education
through personalized learning, intelligent tutoring,
automated assessments, and content generation.
"""

for attempt in range(3):
    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt
        )

        print(response.text.strip())
        break

    except Exception as e:
        print("Error:", e)

        if attempt < 2:
            print("Retrying in 5 seconds...")
            time.sleep(5)
        else:
            print("Failed after 3 attempts.")
```

Error: 401 UNAUTHENTICATED. {'error': {'code': 401, 'message': 'Request had invalid authentication credentials. Expected OAuth 2.0 credential provided by one of the supported authentication providers (https://developers.google.com/identity/protocols/oauth2/devsite)'}. Retrying in 5 seconds...

Error: 401 UNAUTHENTICATED. {'error': {'code': 401, 'message': 'Request had invalid authentication credentials. Expected OAuth 2.0 credential provided by one of the supported authentication providers (https://developers.google.com/identity/protocols/oauth2/devsite)'}. Retrying in 5 seconds...

Error: 401 UNAUTHENTICATED. {'error': {'code': 401, 'message': 'Request had invalid authentication credentials. Expected OAuth 2.0 credential provided by one of the supported authentication providers (https://developers.google.com/identity/protocols/oauth2/devsite)'}. Failed after 3 attempts.

```
from google import genai
import os

client = genai.Client(
    api_key="eneter api key"    #AQ.Ab8RN6K99L0YiYWHaxvTv_ShF98y4ZIx0eS1pliD7gB3lnEwQ
)

prompt = """
### INSTRUCTION ###

Summarize the text in exactly 3 bullet points.

### INPUT TEXT ###

Artificial Intelligence is transforming
education by enabling personalized learning,
automated assessment systems, intelligent
tutoring platforms, and content generation.
It helps students learn at their own pace
while reducing administrative burden on
educators.

### OUTPUT ###
"""
```

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents=prompt  
)  
  
print(response.text)
```

- * AI is transforming education by enabling personalized learning and content generation.
- * It provides automated assessment systems and intelligent tutoring platforms.
- * This allows students to learn at their own pace while reducing administrative burdens for educators.

```
import torch  
import time  
import psutil  
  
from transformers import (  
    pipeline,  
    AutoTokenizer,  
    AutoModel,  
    AutoModelForCausalLM  
)  
  
print("="*60)  
print("HUGGING FACE AI WORKSTATION")  
print("="*60)  
  
# -----  
# Device Information  
# -----  
  
device = "cuda" if torch.cuda.is_available() else "cpu"  
  
print("\nDevice:", device)  
  
if torch.cuda.is_available():  
    print(  
        "GPU:",  
        torch.cuda.get_device_name(0)  
    )  
  
# -----  
# Tokenizer Demo  
# -----  
  
print("\nTOKENIZATION")  
  
tokenizer = AutoTokenizer.from_pretrained(  
    "distilgpt2"  
)  
  
text = ""  
Artificial Intelligence is transforming  
education and cybersecurity.  
""  
  
tokens = tokenizer.tokenize(text)  
  
print("Tokens:")  
print(tokens)  
  
print("Token Count:", len(tokens))  
  
# -----  
# Embedding Demo  
# -----  
  
print("\nEMBEDDINGS")  
  
model = AutoModel.from_pretrained(  
    "bert-base-uncased"  
)  
  
inputs = tokenizer(  
    text,  
    return_tensors="pt"  
)  
  
with torch.no_grad():
```

```

        outputs = model(**inputs)

embedding = outputs.last_hidden_state

print(
    "Embedding Shape:",
    embedding.shape
)

# -----
# Text Generation
# -----

print("\nTEXT GENERATION")

generator = pipeline(
    "text-generation",
    model="distilgpt2"
)

result = generator(
    "Machine Learning is",
    max_new_tokens=50
)

print(
    result[0]["generated_text"]
)

# -----
# Sentiment Analysis
# -----

print("\nSENTIMENT ANALYSIS")

sentiment = pipeline(
    "sentiment-analysis"
)

reviews = [
    "This workshop is amazing.",
    "The session was boring."
]

for review in reviews:

    print(
        sentiment(review)
    )

# -----
# Summarization
# -----

print("\nSUMMARIZATION")

summarizer = pipeline(
    "summarization",
    model="facebook/bart-large-cnn"
)

article = """
Artificial Intelligence is a branch
of computer science focused on creating
systems capable of performing tasks
that typically require human intelligence.
"""

summary = summarizer(
    article,
    max_length=40,
    min_length=10
)

print(
    summary[0]["summary_text"]
)

# -----
# Translation
# -----

print("\nTRANSLATION")

```

```

# -----
# Throughput Benchmark
# -----

print("\nBENCHMARK")

prompt = """
Explain Transformers in AI.
"""

start = time.time()

output = generator(
    prompt,
    max_new_tokens=100
)

end = time.time()

generated_text = output[0]["generated_text"]

tokens_generated = len(
    generated_text.split()
)

throughput = (
    tokens_generated /
    (end-start)
)

print(
    "Time:",
    round(end-start,2),
    "sec"
)

print(
    "Tokens/sec:",
    round(throughput,2)
)

# -----
# Memory Monitoring
# -----

print("\nMEMORY")

process = psutil.Process()

memory = (
    process.memory_info().rss /
    1024**2
)

print(
    "RAM Usage:",
    round(memory,2),
    "MB"
)

# -----
# GPU Monitoring
# -----

if torch.cuda.is_available():

    gpu_memory = (
        torch.cuda.memory_allocated()

```

```

        /1024**3
    )

    print(
        "GPU Memory:",
        round(gpu_memory,2),
        "GB"
    )

print("\nExecution Complete")

```

[Show hidden output](#)

```

from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments
from peft import LoraConfig, get_peft_model
from datasets import Dataset
from transformers import Trainer, DataCollatorForLanguageModeling

# 1. Load a small pretrained model
model_name = "distilgpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(model_name)

# 2. Configure LoRA
lora_config = LoraConfig(
    r=8, # Rank
    lora_alpha=16, # Scaling factor
    lora_dropout=0.1,
    bias="none",
    task_type="CAUSAL_LM"
)

# Apply LoRA
model = get_peft_model(model, lora_config)

# Show trainable parameters
model.print_trainable_parameters()

# 3. Create a small training dataset
texts = [
    "AI is transforming industries.",
    "Machine learning helps computers learn from data.",
    "LoRA enables efficient fine tuning of LLMs.",
    "Large Language Models generate human-like text."
]

dataset = Dataset.from_dict({"text": texts})

# 4. Tokenize data
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=64
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Labels for language modeling
tokenized_dataset = tokenized_dataset.map(
    lambda x: {"labels": x["input_ids"]}
)

# 5. Training arguments
training_args = TrainingArguments(
    output_dir="./lora_output",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    logging_steps=1,
    save_strategy="no"
)

# 6. Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset,
    data_collator=DataCollatorForLanguageModeling(

```

```
data_collator = DataCollatorForLanguageModeling(\n    tokenizer=tokenizer,\n    mlm=False\n)\n\n# 7. Train\ntrainer.train()\n\n# 8. Save LoRA adapter\nmodel.save_pretrained("./lora_adapter")\n\nprint("LoRA fine-tuning completed!")
```